

Year-end Tax Calculator: 설명 가능한 계산 엔진

계산 엔진의 핵심은 결과 숫자가 아니라, 그 숫자에 도달한 규칙과 근거를 설명하는 능력이다.

요약

연말정산 계산기는 표면적으로 입력값을 받아 세액을 계산하는 기능이다. 그러나 실제로는 소득, 공제, 한도, 가족관계, 기간, 증빙, 예외 규칙이 결합된 deterministic rule engine에 가깝다. 이 프로젝트에서는 계산 결과뿐 아니라 어떤 규칙이 어떤 순서로 적용되었는지 trace로 남기는 구조를 중요하게 봤다.

문제 정의

연말정산 영역에서 사용자는 결과 금액만 요구하지 않는다. 왜 그 금액이 나왔는지, 어떤 공제가 적용되었는지, 어떤 항목이 한도에 걸렸는지, 어떤 증빙이 부족한지 확인해야 한다. 운영자 역시 고객 문의에 대응하려면 계산 근거를 설명할 수 있어야 한다.

단순 계산 함수로 구현하면 다음 문제가 발생한다.

- 규칙 변경 시 영향 범위를 추적하기 어렵다.
- 중간 계산 값이 남지 않아 문의 대응이 어렵다.
- 예외 규칙이 코드 곳곳에 흩어진다.
- 같은 입력에 대해 재현 가능한 결과를 보장하기 어렵다.

책임 경계

계층	책임
Input Normalization	소득, 공제, 가족, 증빙 입력을 표준 구조로 변환한다.
Rule Engine	세법·공제·한도 규칙을 deterministic하게 적용한다.
Calculation Trace	적용된 규칙, 중간 값, 제외 사유를 남긴다.
Validation	누락·오류·증빙 부족을 계산 전후에 검증한다.
Output	최종 금액과 설명 가능한 결과를 제공한다.

핵심 설계 결정

계산 과정을 trace로 남겼다

계산 결과만 저장하면 고객 문의에 대응할 수 없다. 따라서 규칙별 입력값, 적용 여부, 중간 결과, 제외 사유를 trace로 남기는 방식이 필요했다.

```
input
-> normalize
-> validate
-> apply income rules
-> apply deduction rules
-> apply limit rules
-> calculate tax
-> emit trace + result
```

규칙 변경을 코드 diff로만 보지 않았다

세법과 공제 기준은 변경될 수 있다. 변경된 규칙이 어떤 입력 케이스에 영향을 주는지 확인하려면 test fixture와 계산 trace가 함께 있어야 한다. 규칙 변경은 단순 코드 수정이 아니라 운영 기준 변경이다.

예외를 숨기지 않았다

계산 엔진에서 예외를 임시 조건문으로 처리하면 시간이 지날수록 검증이 어려워진다. 예외 규칙도 이름과 적용 조건을 가진 rule로 분리해야 한다. 그래야 추적과 테스트가 가능하다.

버린 선택과 이유

연말정산 계산을 화면별 if문으로 처리하는 방식은 제외했다. 세법과 공제 규칙은 매년 바뀌고 예외가 많다. 규칙이 화면 코드에 흩어지면 어떤 기준으로 계산했는지 설명하기 어렵다.

최종 환급액만 저장하는 방식도 충분하지 않았다. 사용자가 결과에 이의를 제기하거나 세법이 변경되었을 때, 당시 어떤 입력과 규칙으로 계산했는지 재현해야 한다. 계산 결과보다 계산 trace가 중요했다.

모든 예외를 자동 보정하는 방식도 피했다. 세무 계산에서는 누락된 증빙, 한도 초과, 적용 불가 항목을 조용히 보정하면 신뢰를 잃는다. 시스템은 예외를 숨기지 않고 사용자가 확인할 수 있는 경고와 근거를 제공해야 했다.

계산 엔진에서 남겨야 하는 기록

기록	목적
rule version	어느 연도·어느 규칙으로 계산했는지 재현한다.
input snapshot	계산 당시 입력값과 이후 수정값을 분리한다.
deduction trace	각 공제 항목의 적용/제외 이유를 설명한다.
limit calculation	한도 적용 전/후 금액을 분리한다.
warning code	사용자 확인이 필요한 항목을 UI와 audit에 남긴다.

운영 검증

검증 기준은 다음과 같았다.

- 같은 입력은 항상 같은 결과를 반환하는가.
- 적용된 공제와 제외된 공제를 설명할 수 있는가.
- 한도 계산의 중간 값을 확인할 수 있는가.
- 규칙 변경 시 regression test가 가능한가.
- 고객 문의에 필요한 trace를 운영자가 확인할 수 있는가.

다른 프로젝트와의 연결

이 경험은 PlanVM과 DBHub 설계에도 영향을 주었다. 계산 규칙과 업무 정책은 코드 내부에 숨기면 운영 설명력이 떨어진다. 반대로 자유롭게 바꾸면 통제력이 약해진다. 따라서 규칙은 deterministic하고 trace 가능한 실행 단위로 관리되어야 한다.

역할과 책임

계산 규칙의 구조화, 입력 normalization, 계산 trace, 예외 규칙 분리, 검증 케이스 기준을 설계했다.

설계 결론

계산 시스템은 수학 함수가 아니라 설명 가능한 업무 엔진이다. 결과보다 중요한 것은 그 결과를 재현하고 설명할 수 있는 구조다.